

TP Python — Mini / ARM

- Durée : 2 heures
 - Niveau : remise à niveau Python — débutant/intermédiaire
 - Travail : individuel ou binôme
-

1. Objectif

Dans ce TP, vous allez construire progressivement MiniARM, un mini-émulateur de processeur, inspiré de l'architecture ARM.

L'architecture du processeur sera très simplifiée. Il possédera :

- 8 registres "R0" à "R7"
- un compteur de programme "PC"
- un pointeur de pile "SP"
- une mémoire sous forme de "bytearray"
- un registre d'état global composé de quelques flags
- un petit jeu d'instructions
- une pile
- un cycle d'exécution Fetch → Decode → Execute

À la fin du TP, vous pourrez faire exécuter à votre processeur des programmes comme :

```
MOV R0, 5
MOV R1, 1
MUL R1, R0, R1
SUB R0, R0, 1
CMP R0, 1
BGT 8
PRINT R1
HALT
```

Le but n'est pas de reproduire exactement un processeur ARM réel.

Le but est de mettre en pratique Python sur un projet concret, et de mobiliser les notions suivantes:

- classes
 - méthodes
 - héritage
 - polymorphisme
 - listes et dictionnaires
 - "bytearray"
 - "struct.pack()" / "struct.unpack()"
 - exceptions
 - tests
 - organisation d'un petit programme
-

2. Architecture de MiniARM

Notre processeur possède :

R0 R1 R2 R3 R4 R5 R6 R7

Chaque registre contient un entier.

Il possède également :

PC : Program Counter
SP : Stack Pointer

PC

“PC” contient l’adresse de la prochaine instruction à exécuter.

SP

“SP” indique le sommet de la pile.

Flags

Nous utiliserons deux flags :

Z : résultat nul
N : résultat négatif

3. La mémoire

La mémoire est un simple tableau d’octets :

```
memory = bytearray(256)
```

Chaque case contient une valeur entre “0” et “255”.

Par exemple :

```
memory[0] = 42  
memory[1] = 100
```

4. Les instructions

Chaque instruction occupe exactement 4 octets :

```
+-----+-----+-----+-----+  
| opcode |  a   |  b   |  c   |  
+-----+-----+-----+-----+
```

Les trois derniers octets dépendent de l’instruction.

Par exemple :

```
MOV R0, 42
```

peut être représenté par :

```
opcode = 1  ---> Opcode de MOV  
a      = 0  ---> Registre de destination R0  
b      = 42 ---> Valeur immédiate  
c      = 0
```

soit :

```
bytes([1, 0, 42, 0])
```

5. Jeu d'instructions

Vous allez implémenter progressivement les instructions suivantes.

Opcode	Instruction	Description
"1"	"MOV Rd, imm"	met une constante dans un registre
"2"	"ADD Rd, Ra, Rb"	addition
"3"	"SUB Rd, Ra, Rb"	soustraction
"4"	"MUL Rd, Ra, Rb"	multiplication
"5"	"CMP Ra, Rb"	compare deux registres
"6"	"B addr"	branchement inconditionnel
"7"	"BEQ addr"	branchement si "Z"
"8"	"BNE addr"	branchement si "Z" est faux
"9"	"PUSH Ra"	empile un registre
"10"	"POP Rd"	dépile vers un registre
"11"	"PRINT Ra"	affiche un registre
"255"	"HALT"	arrête le processeur

En bonus :

12 = BGT

qui branche si le résultat précédent est strictement positif.

6. Organisation des fichiers

Créez :

```
mini-arm/  
├── cpu.py  
├── memory.py  
├── instructions.py  
├── opcodes.py  
└── test_miniarm.py
```

Étape 1 — Les registres

Commencez par créer la classe :

```
class Registers:  
  
    # TODO: implementer pc, sp & les registres généraux rX  
  
class Flags:
```

```
# TODO: implementer Z & N
class CPU:

    def __init__(self):
        self.registers = Registers()
        self.flags = Flags()
        self.running = False
```

Travail

Vérifiez que :

```
cpu = CPU()

print(cpu.registers.r)
print(cpu.registers.pc)
print(cpu.registers.sp)
print(cpu.flags.z)
```

donne quelque chose de cohérent.

Implémentez des méthodes “set” et “get” sur la classe “Registers”, facilitant la manipulation des registres généraux.

Par exemple :

```
cpu.registers.set(0, 42) # R0 = 42
assert cpu.registers.get(0) == 42 # R0 = 42
```

Étape 2 — Les opérations arithmétiques

Ajoutez à “CPU” les méthodes suivantes:

```
def add(self, destination, a, b):
    # ...

def sub(self, destination, a, b):
    # ...

def mul(self, destination, a, b):
    # ...
```

Par exemple :

```
cpu.registers.r[0] = 10
cpu.registers.r[1] = 20

cpu.add(2, 0, 1)

assert cpu.registers.r[2] == 30
```

Puis testez :

```
cpu.sub(3, 1, 0)
assert cpu.registers.r[3] == 10
```

et :

```
cpu.mul(4, 0, 1)
assert cpu.registers.r[4] == 200
```

Étape 3 — Notre première mémoire

Créez :

```
class Memory:

    def __init__(self, size=256):
        self.data = bytearray(size)
```

Ajoutez :

```
def write_byte(self, address, value):
    ...

def read_byte(self, address):
    ...
```

Test :

```
memory = Memory()

memory.write_byte(10, 123)

assert memory.read_byte(10) == 123
```

Question

Pourquoi utilise-t-on ici un “bytearray” plutôt qu’une liste Python ? Pourquoi utiliser “bytearray” et non “bytes” ?

Réfléchissez à ce que représente une mémoire d’ordinateur.

Étape 4 — “struct.pack()” et “struct.unpack()”

Nous voulons maintenant pouvoir stocker des entiers de 32 bits dans notre mémoire. Pour cela, nous allons utiliser les fonctions “pack” et “unpack” du module “struct”.

```
struct.pack(...)
```

transforme une valeur Python en représentation binaire.

```
struct.unpack(...)
```

fait l’opération inverse. Essayez :

```
import struct

value = 123456

data = struct.pack('<I', value)
```

```
print(data)

value2 = struct.unpack('<I', data)[0]

print(value2)
```

Vous devez constater que :

```
value2 == value
```

Étape 5 — Lire et écrire un entier dans la mémoire

Ajoutez à “Memory” :

```
def write_int(self, address, value):
    ...

def read_int(self, address):
    ...
```

Utilisez “struct.pack()” et “struct.unpack()”.

Test :

```
memory.write_int(100, 123456)

assert memory.read_int(100) == 123456
```

Vous aurez probablement besoin de manipuler des slices pour faciliter cet exercice.

Étape 6 — Une instruction

Toutes les instructions vont être représentées par une classe.

Créez :

```
class Instruction:

    def execute(self, cpu):
        raise NotImplementedError
```

Puis créez une première instruction :

```
class Mov(Instruction):

    def __init__(self, destination, value):
        self.destination = destination
        self.value = value

    def execute(self, cpu):
        # ...
```

Elle doit effectuer :

$Rd \leftarrow \text{valeur}$

Test :

```
cpu = CPU()

instruction = Mov(0, 42)
instruction.execute(cpu)

assert cpu.registers[0] == 42
```

Étape 7 — Héritage et polymorphisme

Créez maintenant :

```
class Add(Instruction):
    ...
```

puis :

```
class Sub(Instruction):
    ...
```

et :

```
class Mul(Instruction):
    ...
```

Chaque classe doit implémenter :

```
execute(cpu)
```

Exemple :

```
instructions = [
    Mov(0, 10),
    Mov(1, 20),
    Add(2, 0, 1),
]
```

Puis :

```
for instruction in instructions:
    instruction.execute(cpu)
```

À la fin :

```
assert cpu.registers.r[2] == 30
```

Question importante

Pourquoi cette boucle fonctionne-t-elle sans tester :

```
if instruction is a Mov:
    ...
elif instruction is an Add:
    ...
```

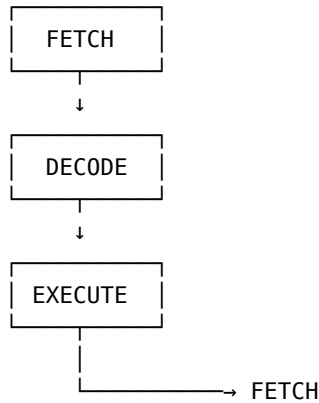
?

Vous venez d'utiliser le polymorphisme.

Étape 8 — Le cycle Fetch → Decode → Execute

Notre CPU doit maintenant pouvoir exécuter des instructions stockées en mémoire.

Le cycle d'un processeur est :



Ajoutez une méthode :

```
def fetch(self):
    ...
```

Elle doit :

1. lire 4 octets à l'adresse "PC" ;
2. avancer "PC" de 4 ;
3. retourner les 4 octets.

Indice

Vous pouvez utiliser :

```
self.memory.data[self.pc:self.pc + 4]
```

Étape 9 — Décoder une instruction

Écrivez :

```
def decode(self, data):
    ...
```

Utilisez :

```
struct.unpack('BBBB', data)
```

Vous obtenez :

opcode, a, b, c

Il faut maintenant transformer ces nombres en objet Python.

Par exemple :

1 0 42 0

doit produire :

```
Mov(0, 42)
```

Indice

Un dictionnaire peut être très utile :

```
instructions = {  
    1: ...,  
    2: ...,  
    3: ...,  
}
```

Vous pouvez également utiliser un “match” :

```
match opcode:  
    case 1:  
        ...  
    case 2:  
        ...
```

Choisissez la solution que vous préférez.

Étape 10 — “run()”

Écrivez maintenant :

```
def run(self):  
    self.running = True  
  
    while self.running:  
        data = self.fetch()  
        instruction = self.decode(data)  
        instruction.execute(self)
```

Il manque encore une instruction essentielle :

```
class Halt(Instruction):  
  
    def execute(self, cpu):  
        ...
```

Elle doit arrêter le processeur.

Étape 11 — Premier programme machine

Créez une fonction permettant d’encoder une instruction :

```
def encode(opcode, a=0, b=0, c=0):  
    ...
```

Elle doit retourner 4 octets.

Par exemple :

```
encode(1, 0, 42, 0)
```

doit retourner :

```
01 00 2A 00
```

Vous pouvez utiliser :

```
bytes([opcode, a, b, c])
```

Premier programme

Construisez le programme :

```
MOV R0, 10
MOV R1, 20
ADD R2, R0, R1
PRINT R2
HALT
```

Chargez-le dans la mémoire.

Puis :

```
cpu.run()
```

Résultat attendu :

```
30
```

Question

Quelle est l'adresse de chaque instruction ?

Étape 12 — Les flags et CMP

Ajoutez :

```
class Cmp(Instruction):
    ...
```

L'instruction :

```
CMP R0, R1
```

calcule conceptuellement :

```
R0 - R1
```

mais ne modifie pas les registres.

Elle modifie seulement :

```
cpu.flags.z
cpu.flags.n
```

Règles :

```
Z = True si résultat == 0
N = True si résultat < 0
```

Exemple :

```
cpu.registers.r[0] = 10
cpu.registers.r[1] = 10
```

```
Cmp(0, 1).execute(cpu)
```

```
assert cpu.flags.z is True
```

Étape 13 — Les branchements

Ajoutez :

```
class Branch(Instruction):
    ...
```

Elle réalise :

```
cpu.registers.pc = address
```

Puis :

```
class Beq(Instruction):
    ...
```

Elle réalise :

```
if cpu.flags.z:
    cpu.registers.pc = address
```

Enfin :

```
class Bne(Instruction):
    ...
```

qui branche lorsque “Z” est faux.

Étape 14 — Tester une condition

Construisez un programme qui réalise :

```
R0 = 10
R1 = 10
```

```
si R0 == R1:
    R2 = 42
```

```
sinon:
    R2 = 99
```

Vous pouvez utiliser :

```
MOV
CMP
BEQ
MOV
HALT
```

Attention : les adresses sont des adresses mémoire, donc :

```
0
4
8
12
16
...
```

Étape 15 — La pile

Ajoutez au CPU :

```
def push(self, value):
    ...
```

```
def pop(self):
    ...
```

La pile doit utiliser “SP”.

Pour cette version simplifiée :

PUSH :

SP diminue
valeur écrite à SP

et :

POP :

valeur lue à SP
SP augmente

Vous pouvez stocker les valeurs avec :

```
memory.write_int(...)
memory.read_int(...)
```

Tester la pile

```
cpu.push(123)
cpu.push(456)
```

```
assert cpu.pop() == 456
assert cpu.pop() == 123
```

Question

Pourquoi le deuxième “pop()” retourne-t-il “123” ?

Quel type de structure de données est une pile ?

Étape 16 — PUSH et POP comme instructions

Créez :

```
class Push(Instruction):
    ...
```

et :

```
class Pop(Instruction):
    ...
```

Exemple :

```
MOV R0, 42
PUSH R0
MOV R0, 100
POP R1
```

À la fin :

```
R0 = 100
R1 = 42
```

Étape 17 — Challenge final

Si vous avez terminé les étapes précédentes, essayez de faire fonctionner ce programme :

```
MOV R0, 5
MOV R1, 1

loop:
    MUL R1, R0, R1
    SUB R0, R0, 1
    CMP R0, R2
    BGT loop
```

```
PRINT R1
HALT
```

Vous devrez déterminer comment représenter :

R2 = 1

et comment implémenter :

BGT

Objectif

Le programme doit calculer :

$5! = 120$

et afficher :

120

Bonus — “BGT”

Ajoutez l’instruction :

```
class Bgt(Instruction):  
    ...
```

Elle doit effectuer le branchement lorsque le résultat du dernier “CMP” est strictement positif.

Une implémentation simplifiée peut utiliser :

```
if not cpu.flags.z and not cpu.flags.n:  
    cpu.registers.pc = address
```

Bonus ++ — Écrire un mini assembleur

Si vous avez encore du temps, essayez de transformer :

```
MOV R0, 10  
MOV R1, 20  
ADD R2, R0, R1  
PRINT R2  
HALT
```

en programme machine automatiquement.

L’objectif serait de pouvoir écrire :

```
program = assemble('''  
MOV R0, 10  
MOV R1, 20  
ADD R2, R0, R1  
PRINT R2  
HALT  
''')
```

puis :

```
cpu.load(program)  
cpu.run()
```

Tests à avoir à la fin

Votre projet devrait au minimum permettre de vérifier :

```
# MOV
cpu = CPU()
Mov(0, 42).execute(cpu)
assert cpu.registers.r[0] == 42
```

```
# ADD
cpu.registers.r[0] = 10
cpu.registers.r[1] = 20

Add(2, 0, 1).execute(cpu)

assert cpu.registers.r[2] == 30
```

```
# CMP
cpu.registers.r[0] = 10
cpu.registers.r[1] = 10

Cmp(0, 1).execute(cpu)

assert cpu.flags.z
```

```
# Stack
cpu.push(123)
cpu.push(456)

assert cpu.pop() == 456
assert cpu.pop() == 123
```

Et surtout :

Un programme complet doit pouvoir être chargé en mémoire puis exécuté avec `cpu.run()`.

Bilan

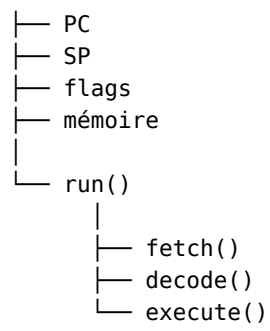
À la fin du TP, vous devriez être capable d’expliquer :

1. Qu’est-ce qu’un registre ?
 2. À quoi sert “PC” ?
 3. À quoi sert “SP” ?
 4. Pourquoi utiliser un “bytearray” pour représenter la mémoire ?
 5. À quoi servent “pack()” et “unpack()” ?
 6. Comment une instruction devient-elle un objet Python ?
 7. Où intervient l’héritage ?
 8. Qu’est-ce que le polymorphisme dans ce projet ?
 9. Comment fonctionne le cycle Fetch → Decode → Execute ?
 10. Comment un branchement permet-il de créer une boucle ?
 11. Pourquoi une pile fonctionne-t-elle en LIFO ?
-

Objectif minimum

Si vous êtes arrivé jusqu’ici :

CPU
└─ registres



vous avez construit votre propre mini-machine virtuelle en Python.